



Error Detection and Correction



Note

**Data can be corrupted
during transmission.**

**Some applications require that
errors be detected and corrected.**

Figure 10.1 *Single-bit error*

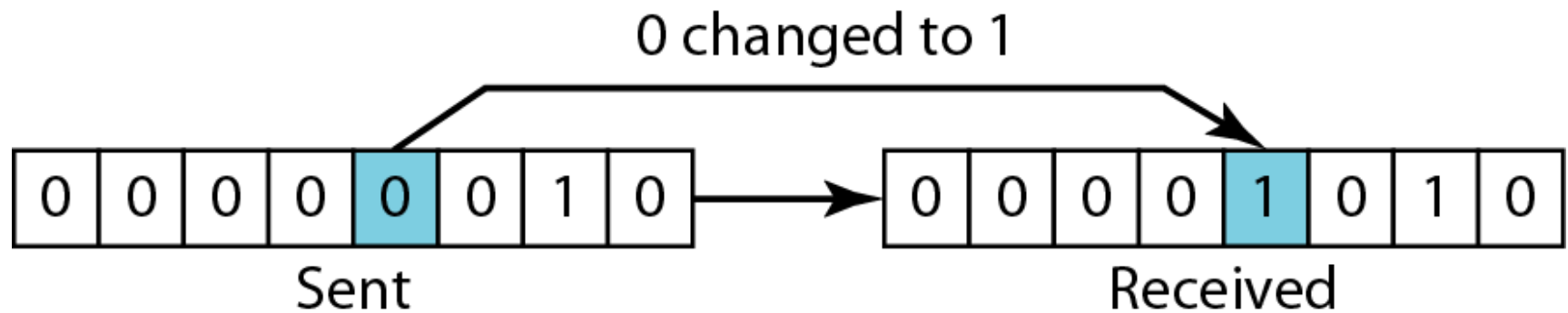
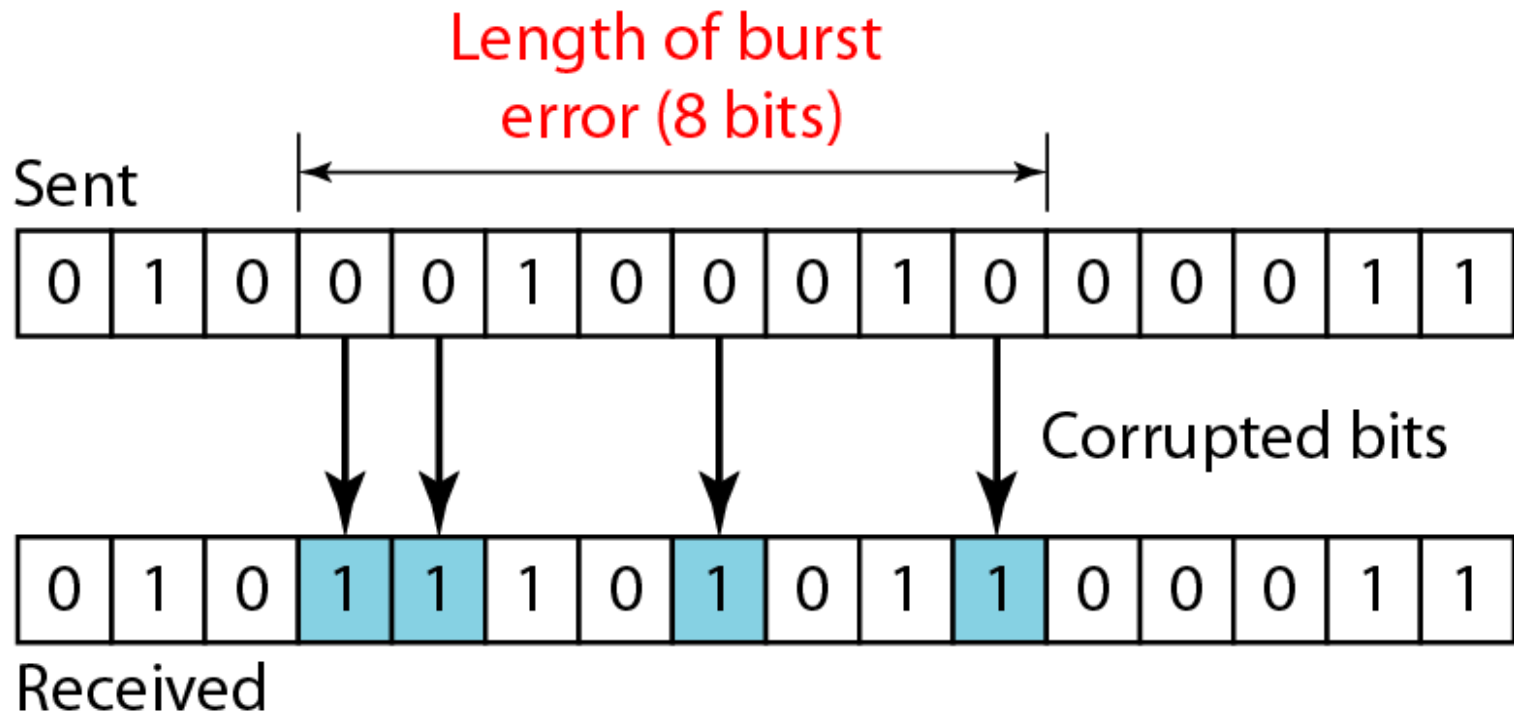


Figure 10.2 *Burst error of length 8*





Note

To detect or correct errors, we need to send extra (redundant) bits with data.

Figure 10.3 *The structure of encoder and decoder*

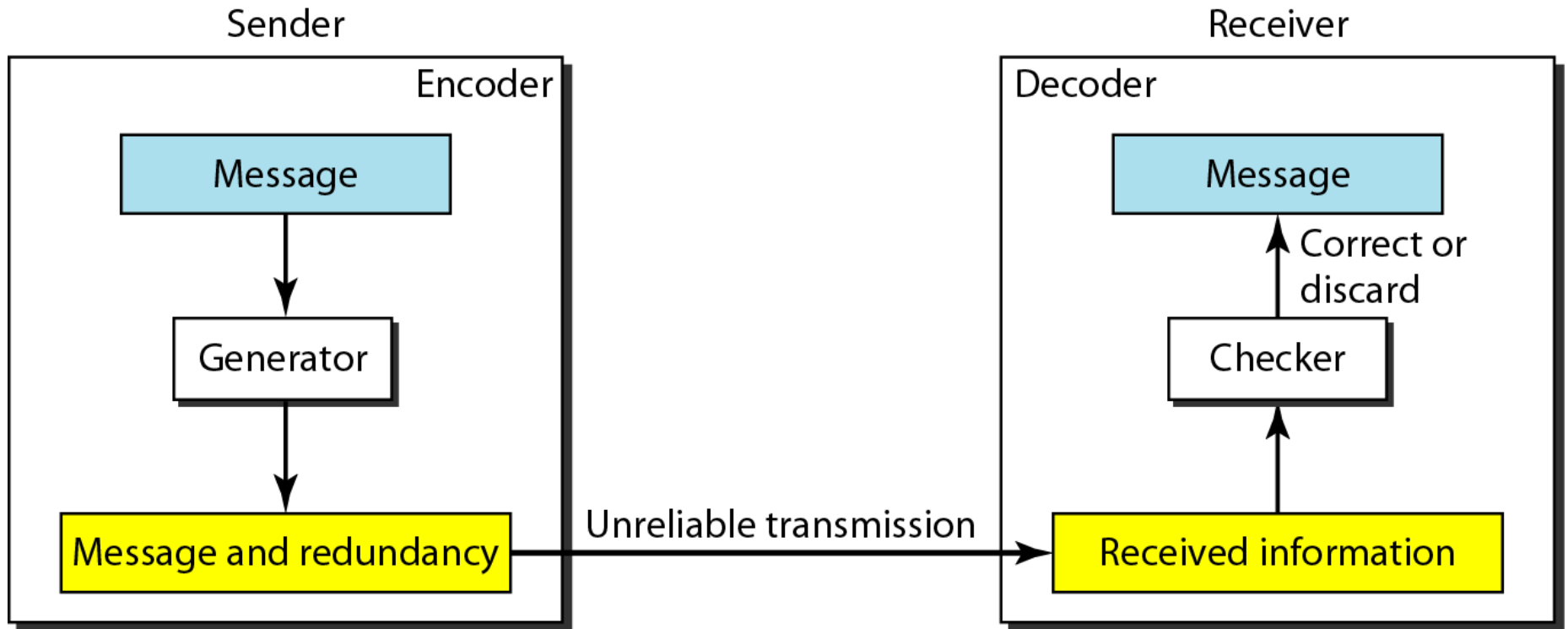


Figure 10.4 *XORing of two single bits or two words*

$$0 \oplus 0 = 0 \qquad 1 \oplus 1 = 0$$

a. Two bits are the same, the result is 0.

$$0 \oplus 1 = 1 \qquad 1 \oplus 0 = 1$$

b. Two bits are different, the result is 1.

	1	0	1	1	0
$\oplus$	1	1	1	0	0
	0	1	0	1	0

c. Result of XORing two patterns

10-2 BLOCK CODING

*In block coding, we divide our message into blocks, each of k bits, called **datawords**. We add r redundant bits to each block to make the length $n = k + r$. The resulting n -bit blocks are called **codewords**.*

Topics discussed in this section:

Error Detection

Error Correction

Hamming Distance

Minimum Hamming Distance

Figure 10.5 *Datawords and codewords in block coding*

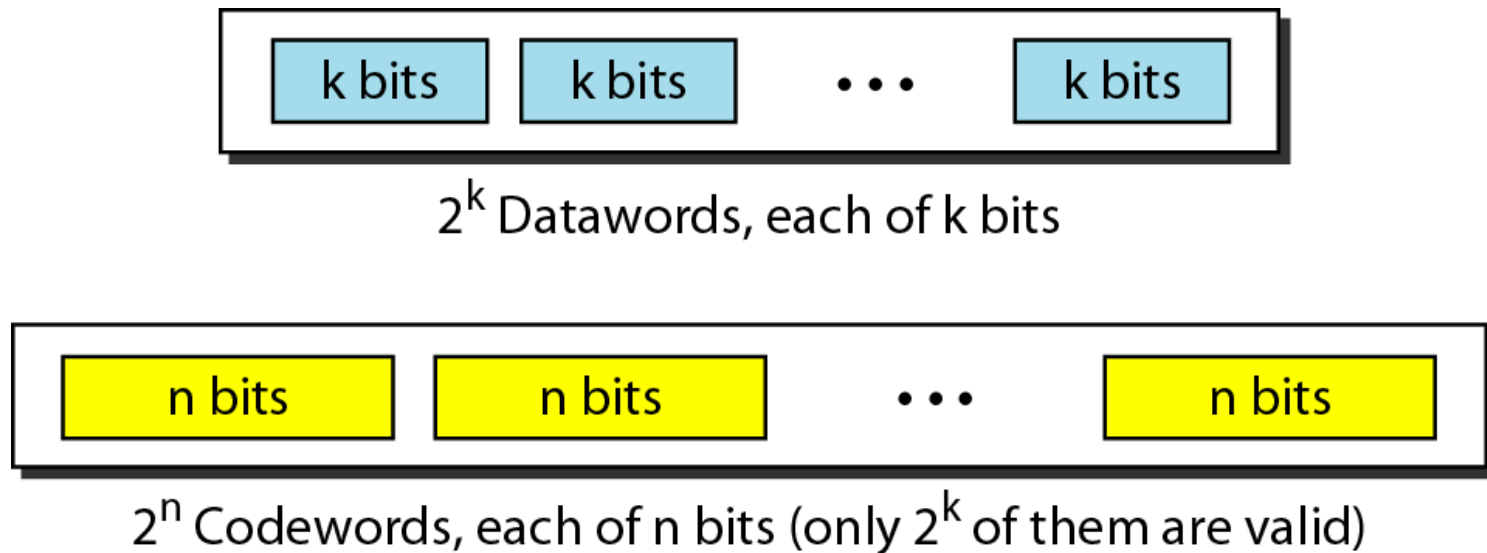


Figure 10.6 *Process of error detection in block coding*

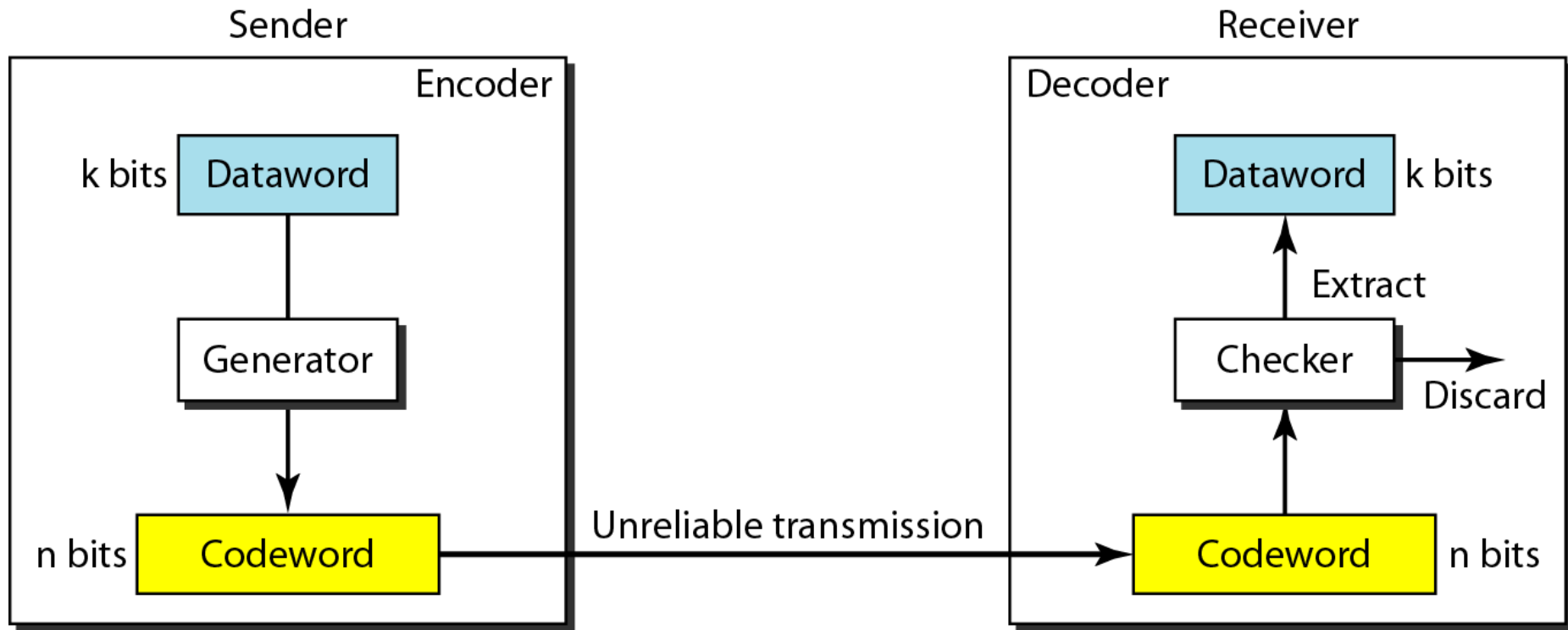


Table 10.1 *A code for error detection (Example 10.2)*

<i>Datawords</i>	<i>Codewords</i>
00	000
01	011
10	101
11	110

Note

An error-detecting code can detect only the types of errors for which it is designed; other types of errors may remain undetected.

Note

The Hamming distance between two words is the number of differences between corresponding bits.



Example 10.4

Let us find the Hamming distance between two pairs of words.

1. *The Hamming distance $d(000, 011)$ is 2 because*

$$000 \oplus 011 \text{ is } 011 \text{ (two 1s)}$$

2. *The Hamming distance $d(10101, 11110)$ is 3 because*

$$10101 \oplus 11110 \text{ is } 01011 \text{ (three 1s)}$$

Note

The minimum Hamming distance is the smallest Hamming distance between all possible pairs in a set of words.



Example 10.5

Find the minimum Hamming distance of the coding scheme in Table 10.1.

Solution

We first find all Hamming distances.

$d(000, 011) = 2$	$d(000, 101) = 2$	$d(000, 110) = 2$	$d(011, 101) = 2$
$d(011, 110) = 2$	$d(101, 110) = 2$		

The $d_{\min}$ in this case is 2.



Example 10.6

Find the minimum Hamming distance of the coding scheme in Table 10.2.

Solution

We first find all the Hamming distances.

$d(00000, 01011) = 3$	$d(00000, 10101) = 3$	$d(00000, 11110) = 4$
$d(01011, 10101) = 4$	$d(01011, 11110) = 3$	$d(10101, 11110) = 3$

The $d_{\min}$ in this case is 3.

Note

To guarantee the detection of up to s errors in all cases, the minimum Hamming distance in a block code must be $d_{\min} = s + 1$.



Example 10.7

The minimum Hamming distance for our first code scheme (Table 10.1) is 2. This code guarantees detection of only a single error. For example, if the third codeword (101) is sent and one error occurs, the received codeword does not match any valid codeword. If two errors occur, however, the received codeword may match a valid codeword and the errors are not detected.



Example 10.8

Our second block code scheme (Table 10.2) has $d_{\min} = 3$. This code can detect up to two errors. Again, we see that when any of the valid codewords is sent, two errors create a codeword which is not in the table of valid codewords. The receiver cannot be fooled.

However, some combinations of three errors change a valid codeword to another valid codeword. The receiver accepts the received codeword and the errors are undetected.

Note

To guarantee correction of up to t errors in all cases, the minimum Hamming distance in a block code must be $d_{\min} = 2t + 1$.



Example 10.9

A code scheme has a Hamming distance $d_{\min} = 4$. What is the error detection and correction capability of this scheme?

Solution

*This code guarantees the detection of up to **three** errors ($s = 3$), but it can correct up to **one** error. In other words, if this code is used for error correction, part of its capability is wasted. Error correction codes need to have an odd minimum distance (3, 5, 7, ...).*

Note

**A simple parity-check code is a
single-bit error-detecting
code in which
 $n = k + 1$ with $d_{\min} = 2$.**

Table 10.3 *Simple parity-check code $C(5, 4)$*

<i>Datawords</i>	<i>Codewords</i>	<i>Datawords</i>	<i>Codewords</i>
0000	00000	1000	10001
0001	00011	1001	10010
0010	00101	1010	10100
0011	00110	1011	10111
0100	01001	1100	11000
0101	01010	1101	11011
0110	01100	1110	11101
0111	01111	1111	11110



Note

**A simple parity-check code can detect
an odd number of errors.**

Figure 10.11 *Two-dimensional parity-check code*

1	1	0	0	1	1	1	1	Row parities
1	0	1	1	1	0	1	1	
0	1	1	1	0	0	1	0	
0	1	0	1	0	0	1	1	
Column parities								
0	1	0	1	0	1	0	1	

a. Design of row and column parities

Figure 10.15 *Division in CRC encoder*

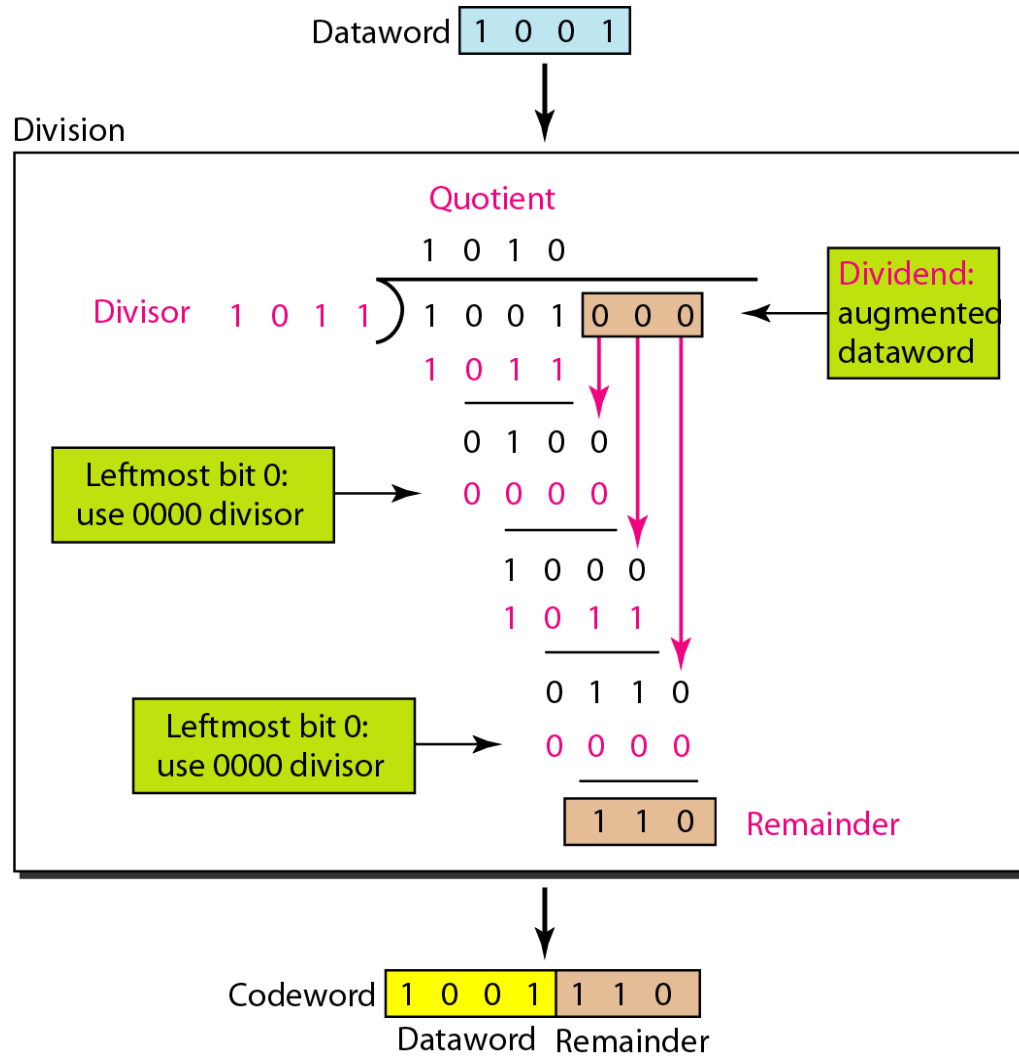


Figure 10.16 *Division in the CRC decoder for two cases*

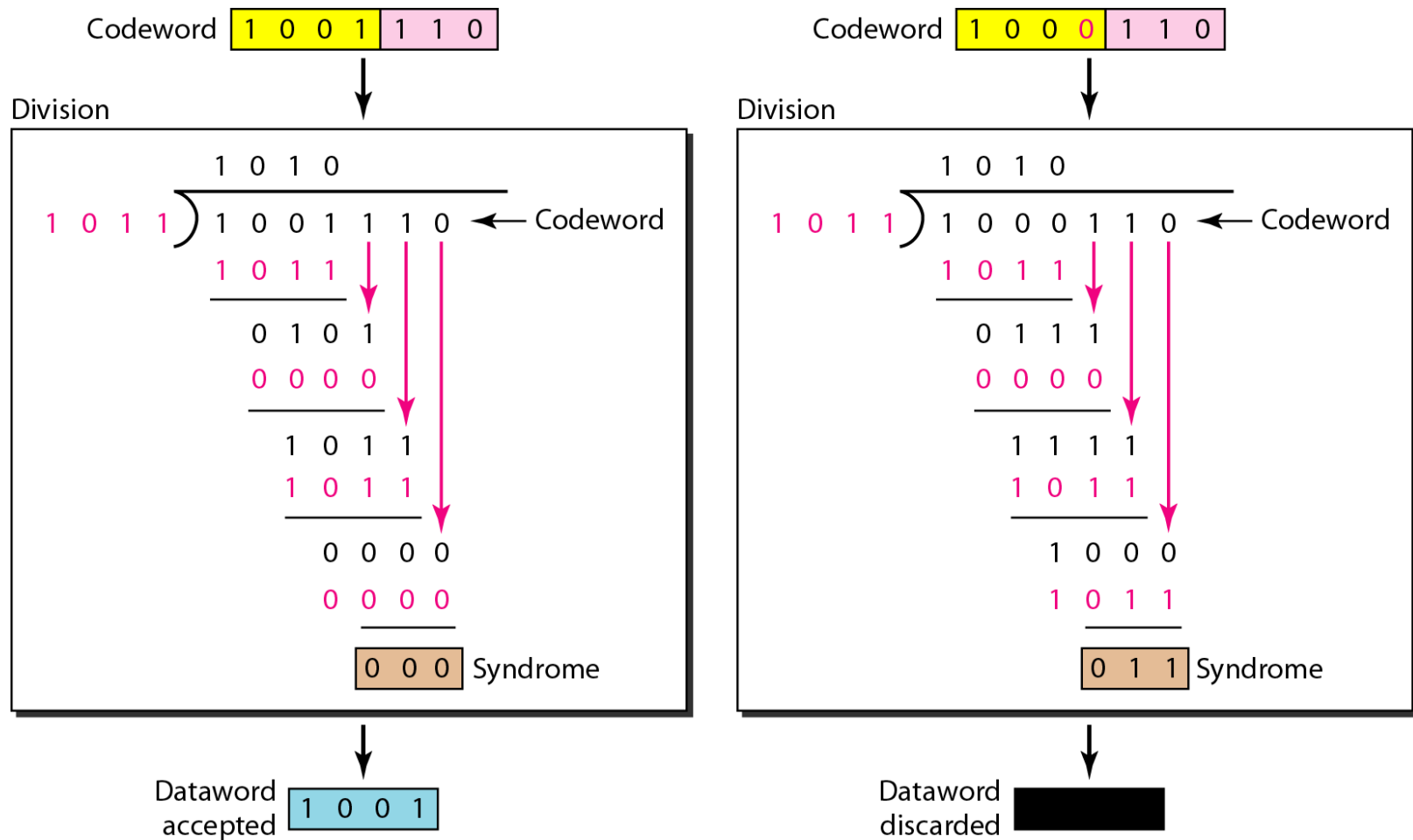


Figure 10.18 *Simulation of division in CRC encoder*

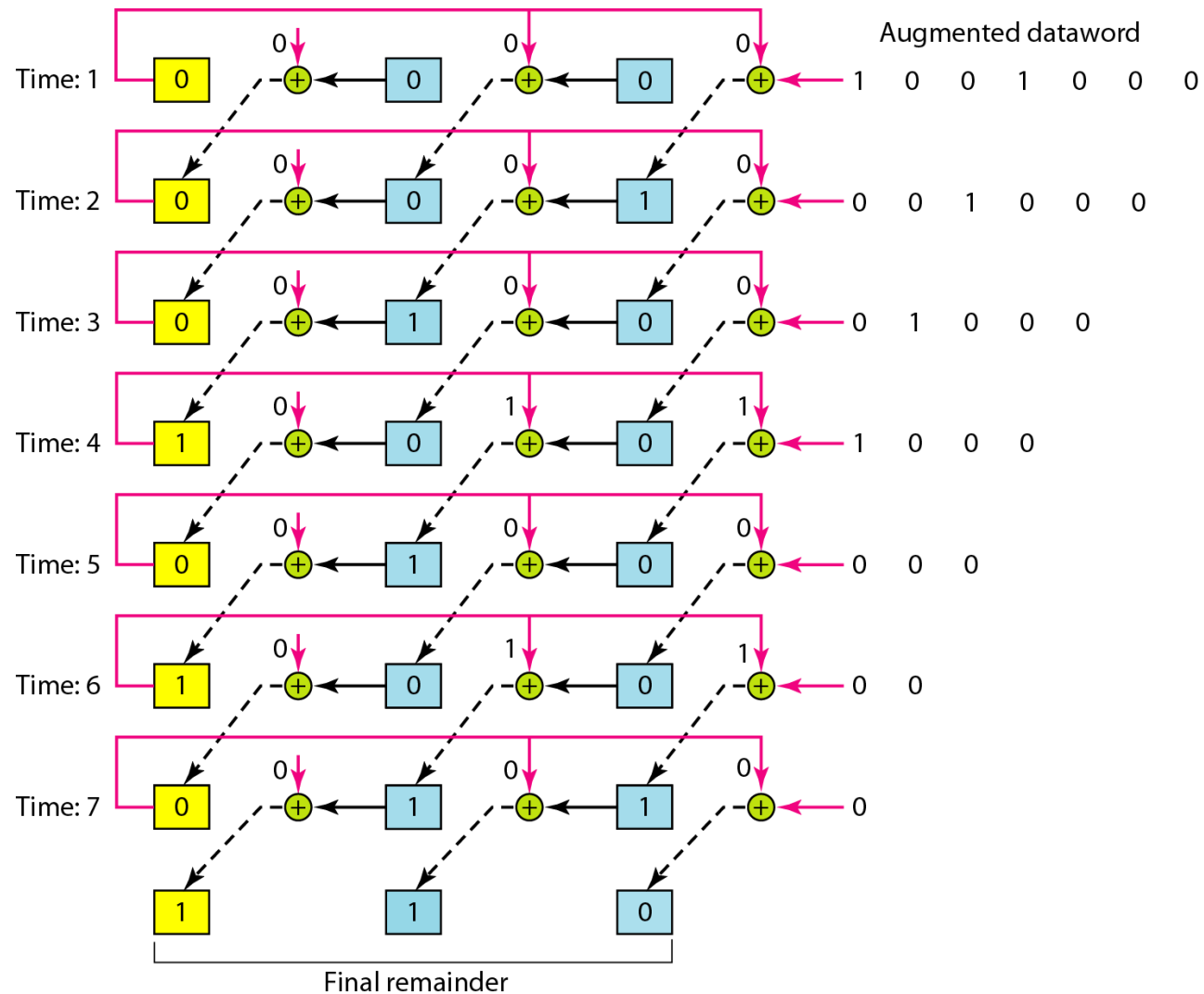


Figure 10.19 *The CRC encoder design using shift registers*

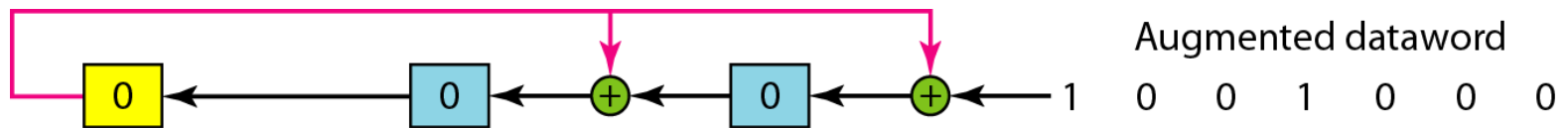
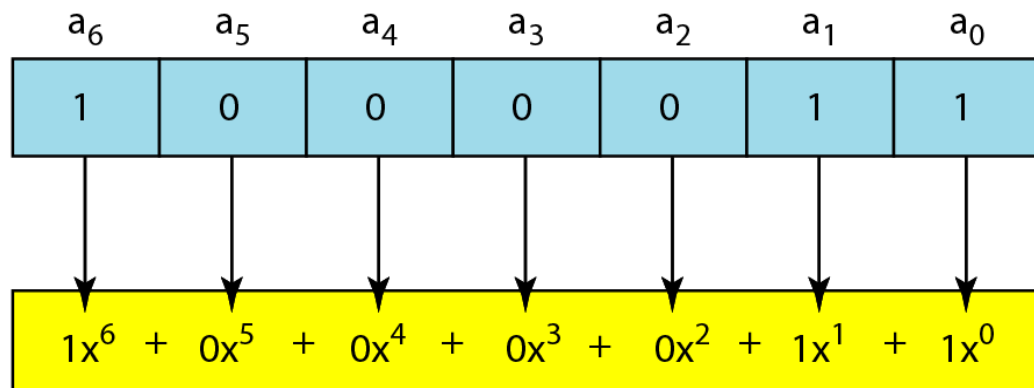
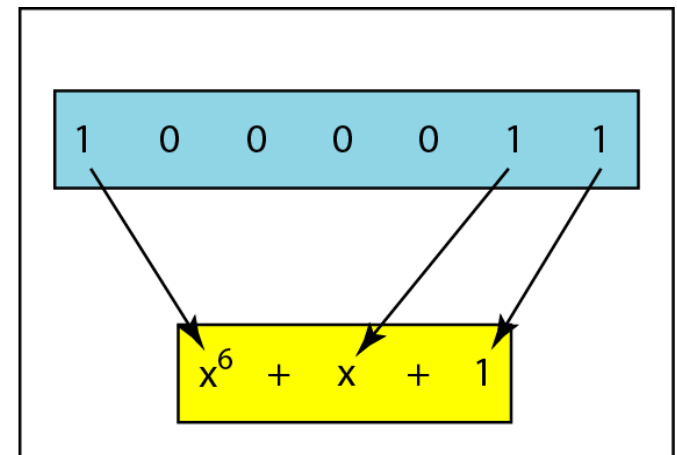


Figure 10.21 *A polynomial to represent a binary word*

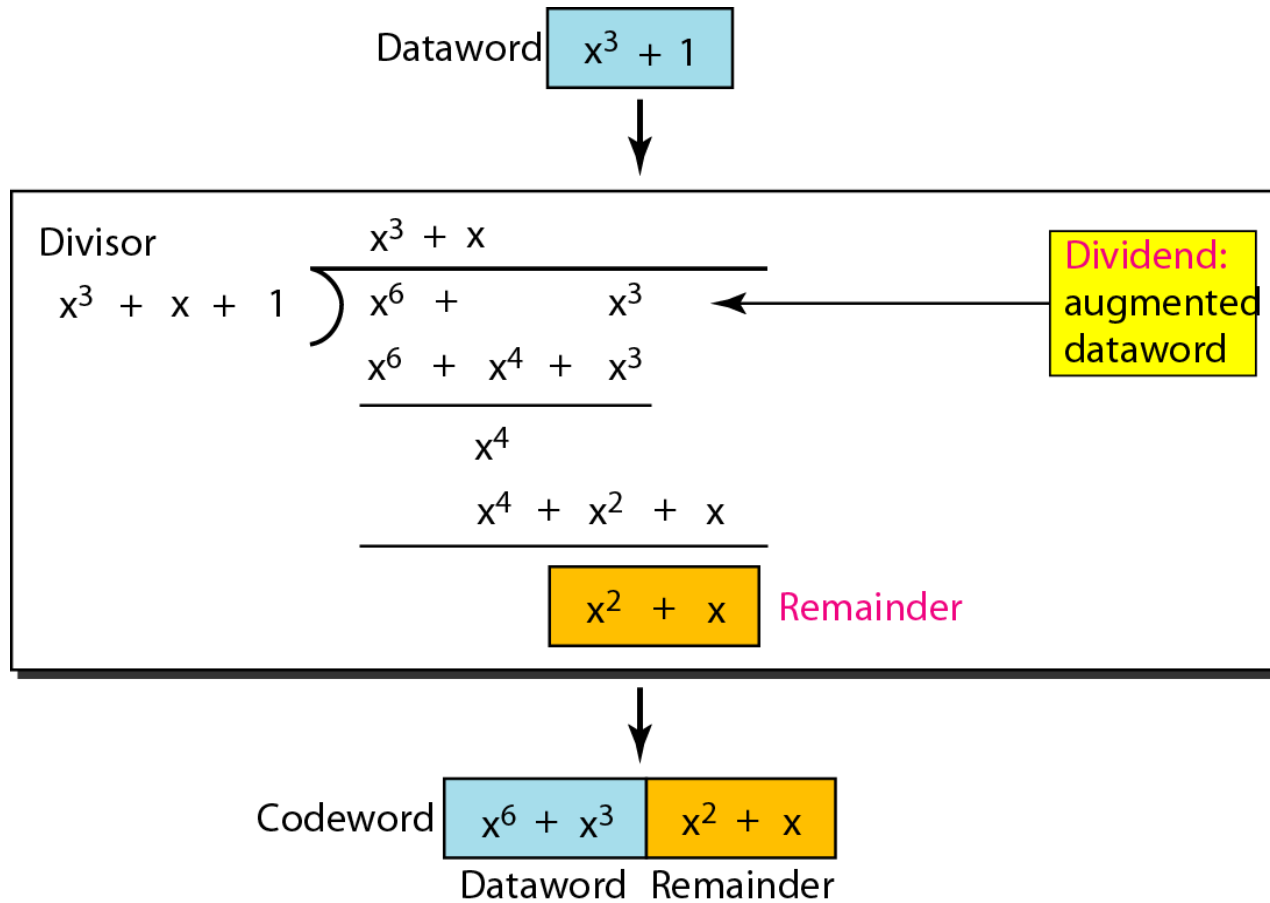


a. Binary pattern and polynomial



b. Short form

Figure 10.22 *CRC division using polynomials*



Note

The divisor in a cyclic code is normally called the generator polynomial or simply the generator.

Note

In a cyclic code,

If $s(x) \neq 0$, one or more bits is corrupted.

If $s(x) = 0$, either

- a. No bit is corrupted. or**
- b. Some bits are corrupted, but the decoder failed to detect them.**

Note

In a cyclic code, those $e(x)$ errors that are divisible by $g(x)$ are not caught.

Note

**If the generator has more than one term
and the coefficient of x^0 is 1,
all single errors can be caught.**

Note

A generator that contains a factor of $x + 1$ can detect all odd-numbered errors.

Note

A good polynomial generator needs to have the following characteristics:

- 1. It should have at least two terms.**
- 2. The coefficient of the term x^0 should be 1.**
- 3. It should not divide $x^t + 1$, for t between 2 and $n - 1$.**
- 4. It should have the factor $x + 1$.**

10-5 CHECKSUM

The last error detection method we discuss here is called the checksum. The checksum is used in the Internet by several protocols although not at the data link layer. However, we briefly discuss it here to complete our discussion on error checking

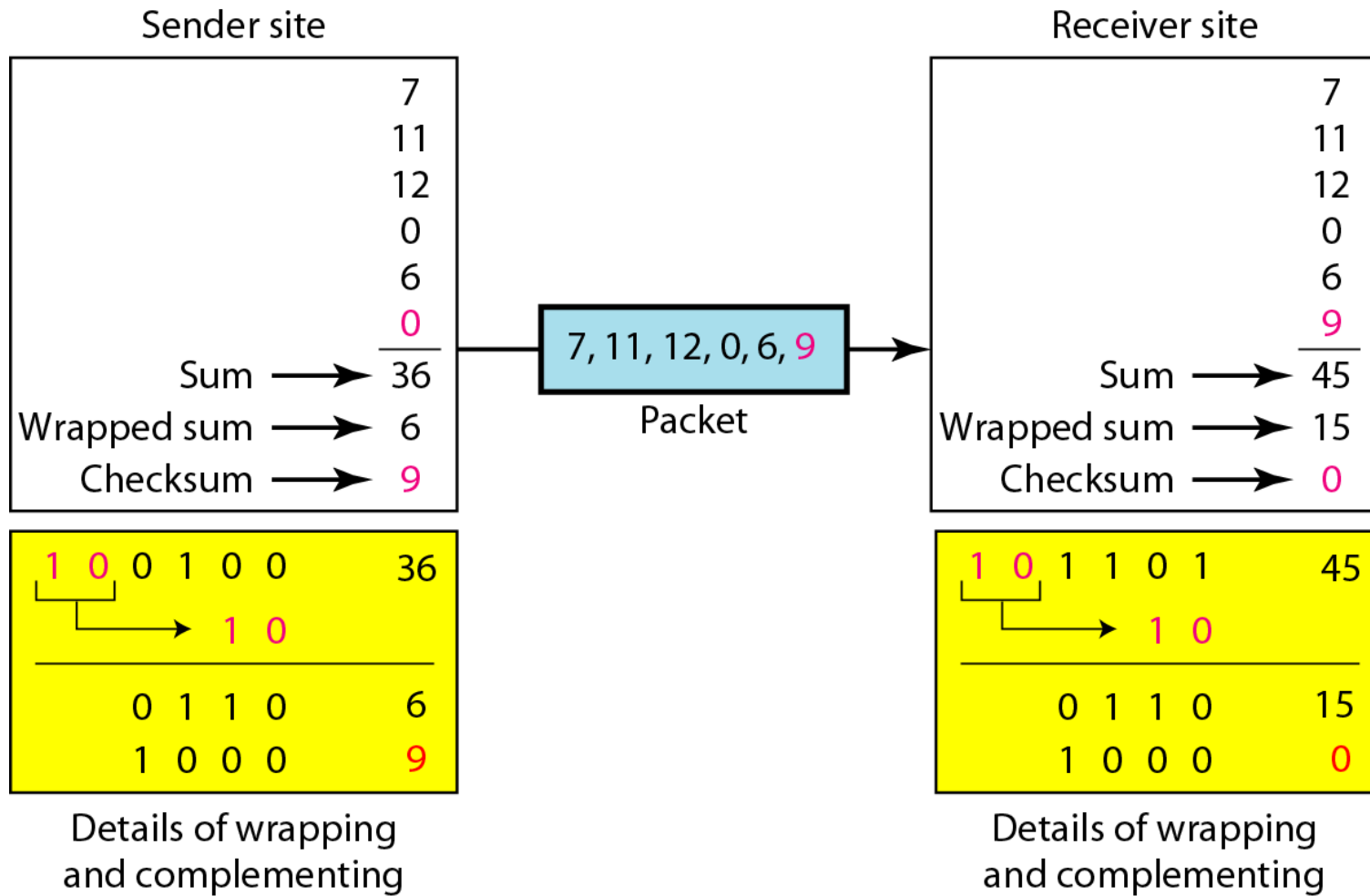
Topics discussed in this section:

Idea

One's Complement

Internet Checksum

Figure 10.24 *Example 10.22*





Note

Sender site:

- 1. The message is divided into 16-bit words.**
- 2. The value of the checksum word is set to 0.**
- 3. All words including the checksum are added using one's complement addition.**
- 4. The sum is complemented and becomes the checksum.**
- 5. The checksum is sent with the data.**



Note

Receiver site:

- 1. The message (including checksum) is divided into 16-bit words.**
- 2. All words are added using one's complement addition.**
- 3. The sum is complemented and becomes the new checksum.**
- 4. If the value of checksum is 0, the message is accepted; otherwise, it is rejected.**

Hamming Code for Error Detection & Correction

Position	7	6	5	4	3	2	1
Bit	d_3	d_2	d_1	p_2	d_0	p_1	p_0

$$p_2 = d_3 \oplus d_2 \oplus d_1$$

$$p_1 = d_3 \oplus d_2 \oplus d_0$$

$$p_0 = d_3 \oplus d_1 \oplus d_0$$

Example of redundancy bit calculation

d	d	d	r8	d	d	d	r4	d	r2	r1
---	---	---	----	---	---	---	----	---	----	----

$r1=1,3,5,7,9,11$

$r2=2,3,6,7,10,11$

Input Data 1001101

$r4=4,5,6,7$

$r8=8,9,10,11$

Adding r1

11	10	9	8	7	6	5	4	3	2	1
1	0	0	r8	1	1	0	r4	1	r2	1

Adding r2

11	10	9	8	7	6	5	4	3	2	1
1	0	0	r8	1	1	0	r4	1	0	1

Adding r4

11	10	9	8	7	6	5	4	3	2	1
1	0	0	r8	1	1	0	0	1	0	1

Adding r8

11	10	9	8	7	6	5	4	3	2	1
1	0	0	1	1	1	0	0	1	0	1

Output data: 10011100101